



Automated Layout Generation and Verification

Madhava Sriram (22b1233)

Supervisor: Prof. Maryam Shojaei Baghini

Mentors: Moin Shaikh and Prajwal



Problem Statement Overview

- Given a schematic (in my case a rectifier consisting of 2 mosfets and a capacitor), we should be able to automate layout creation and also create optimised versions of it.
- This would help to reduce the burden on people who spend humongous amount of time to create layout, observe desirables and reiterate the process, only to find slightly better layout. This process can now be automated and done at comparatively faster pace.
- Objective is to perform better than/at-par with manually made layout.

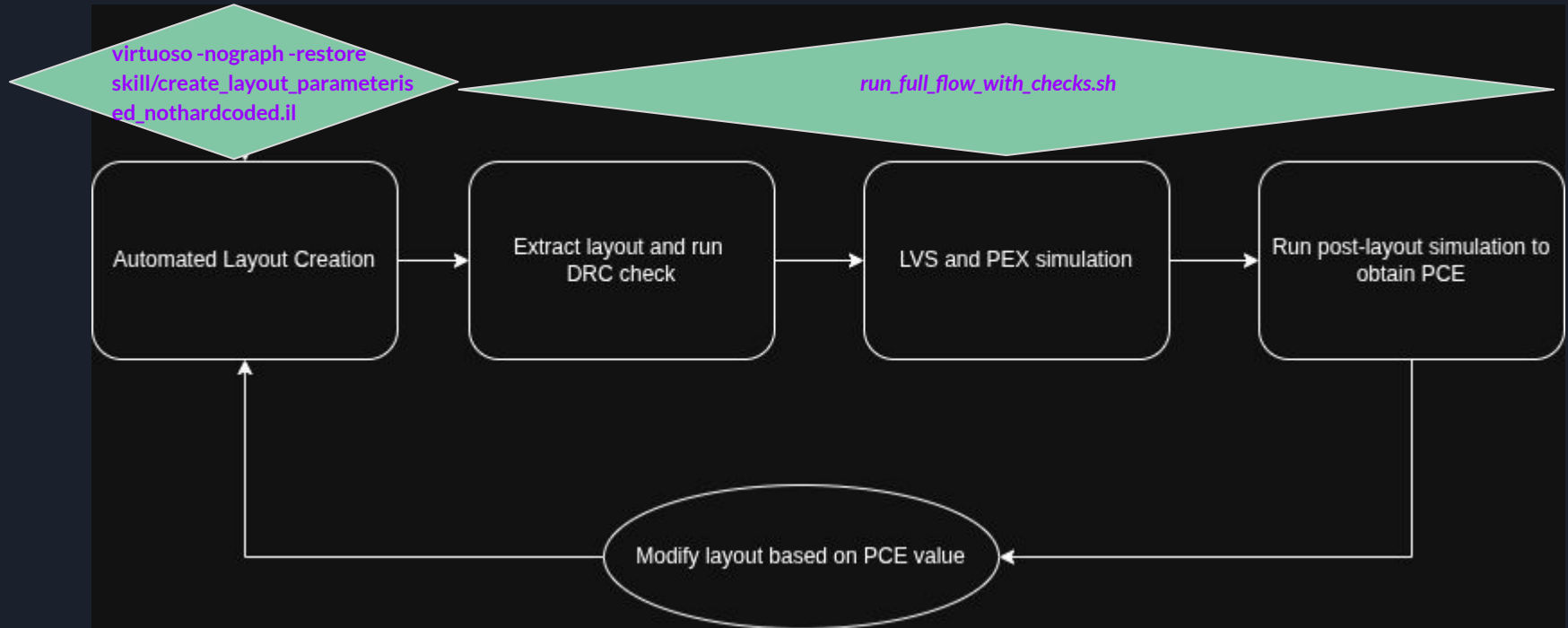


Overview of the tools/terms used in the flow:

- **SKILL Scripting Engine:**
It is Cadence's internal automation scripting language. SKILL is used to automatically generate an inductor layout, avoiding manual drawing. SKILL also interfaces with verification stage of the layout.
- **Power Conversion Efficiency (PCE):**
Power Conversion Efficiency (PCE) is the ratio of useful output power to input power, expressed as a percentage to indicate how efficiently energy is converted.
- **DRC (Design Rule Check):** Verifies that the physical layout follows the manufacturing/fabrication design rules.
- **LVS (Layout Versus Schematic):** Checks whether the extracted layout netlist matches the original schematic/netlist electrically.
- **PEX (Parasitic Extraction):** Extracts parasitic resistances, capacitances (and sometimes inductances) from the layout for accurate post-layout simulation.

Flow diagram of the process...

Note: All of this is through skill code and no step of this involves Virtuoso GUI

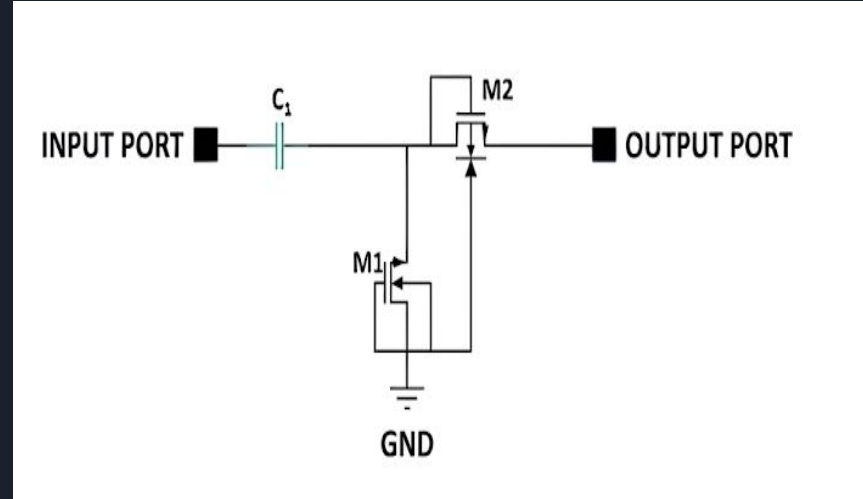
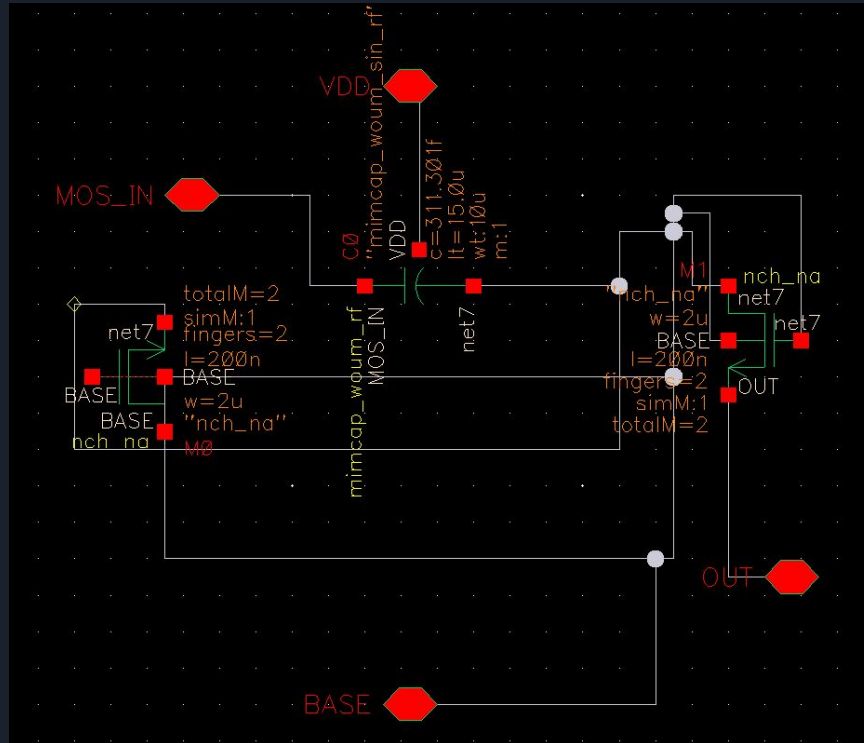




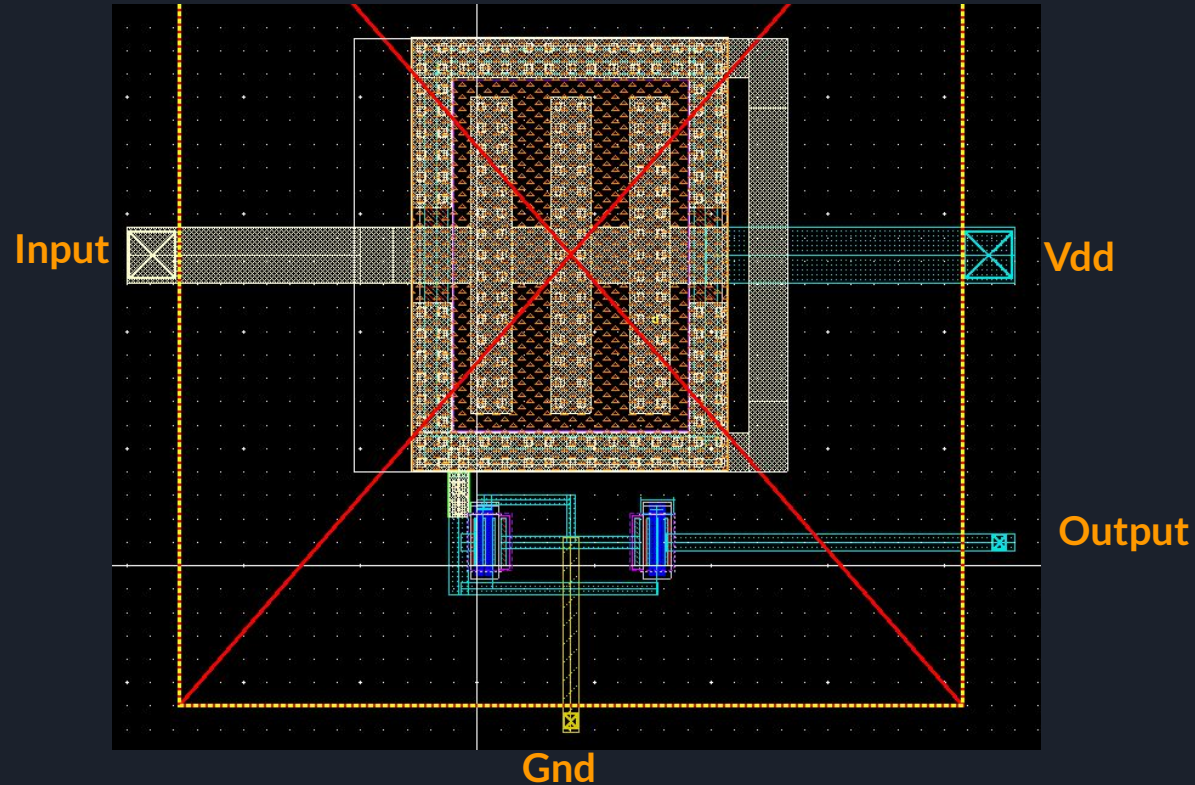
Initial Approach and Thought Process

- Initially, the main focus was to complete the pipeline of layout creation
- So, the layout was hardcoded to the coordinates level and the idea was to move elements relative to each other to vary the layout (similar to scaling the layout in a certain direction)
- Rest of the flow (ie; DRC, LVS, PEX and post layout simulation), had to be completed to keep only the layout creation as the variable component.

The Schematic



Initial layout generated through pure hard-coding
of location of components and wires

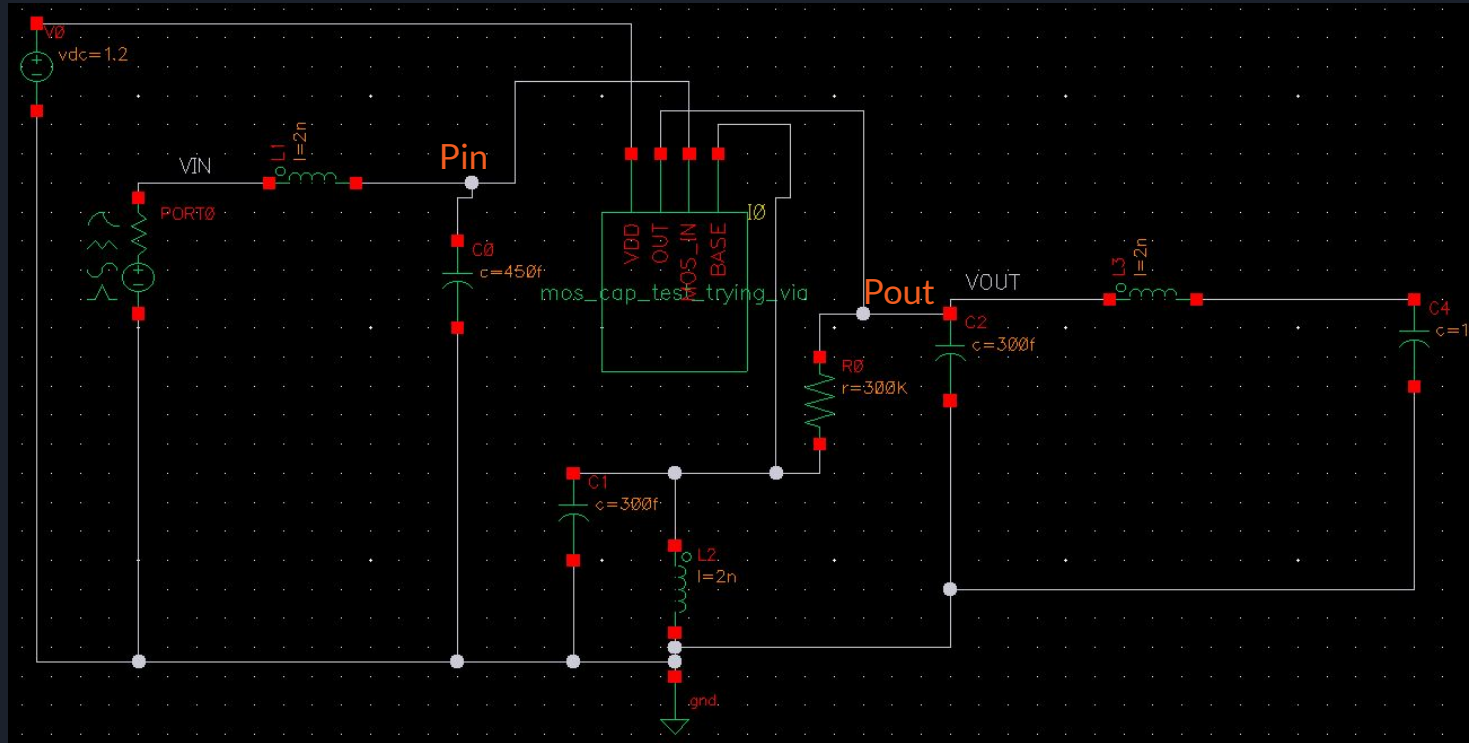




Verifying first through GUI...

- The correctness of layout (ie; DRC and LVS checks) was done initially through GUI to first show in-practice that entire workable layout can be generated through SKILL coding
- Through GUI, I first made sure that there were no DRC errors, no LVS errors and PEX errors, and ran the post-layout simulation

Test Bench



GUI simulation result:

- For simulation we also include other extra external parasitic inductances and capacitances, so the input we give to the setup and Pin of the rectifier are different, Pin is the power given to the rectifier, whereas the input we give is to the rectifier + external parasitics setup
- Our region of interest is the point where Pin is -30dBm.
- We see that PCE is around 46%, and is very much similar to what my mentor, Prajwal, got when he had run the simulation on entirely different layout he had created and tested.





Automating layout verification process:

- The file [run_full_flow_with_checks.sh](#) is the main file in this context.
- It extracts the layout, runs DRC check, LVS and PEX and post layout simulation and outputs the value of PCE corresponding to the Pin we want to observe.
- Here, we are sweeping input within a range, so the Pin that we want might not be present in swept values, so look for Pin present in swept values and look for value closest to -30dBm.

```
[madhava@vlsi33 work_tsmc_gp_65_2f]$ ./run_full_flow_with_checks.sh
Step 1: Running DRC... Done (Verified known violations).
Step 2a: Running LVS... Done (LVS Clean).
Step 2b: Running PEX... Done (Netlist synced).
Step 3: Running Simulation... Done.

-----
Target Pin: -30.00 dBm | Actual Pin: -29.87 dBm
FINAL PCE: 47.22%
-----
```



Layout modification stage:

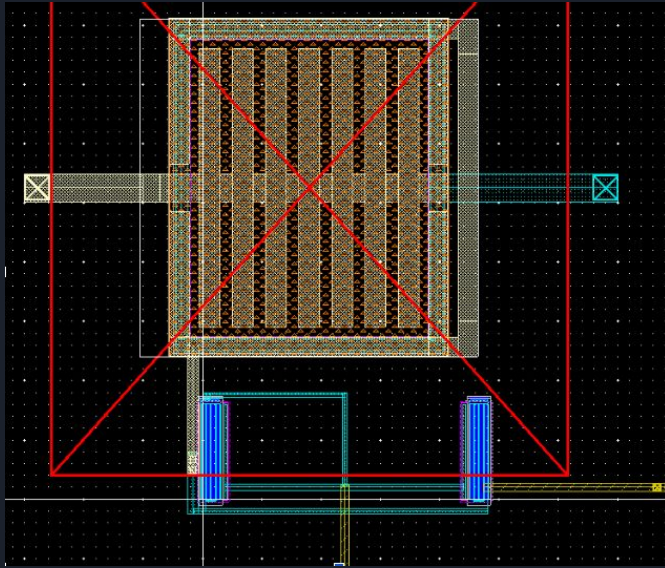
- Now that pipeline is completed, as we had expected, now we need to only modify layout and we can directly check in automated fashion if the layout is an improvement over the previous or not.
- First stage of generalisation implemented to remove hard-coding was including these params that are changeable:

LENGTH_OF_MOSFET
WIDTH_OF_MOSFET
LENGTH_OF_CAP
WIDTH_OF_CAP
NUMBER_OF_FINGERS

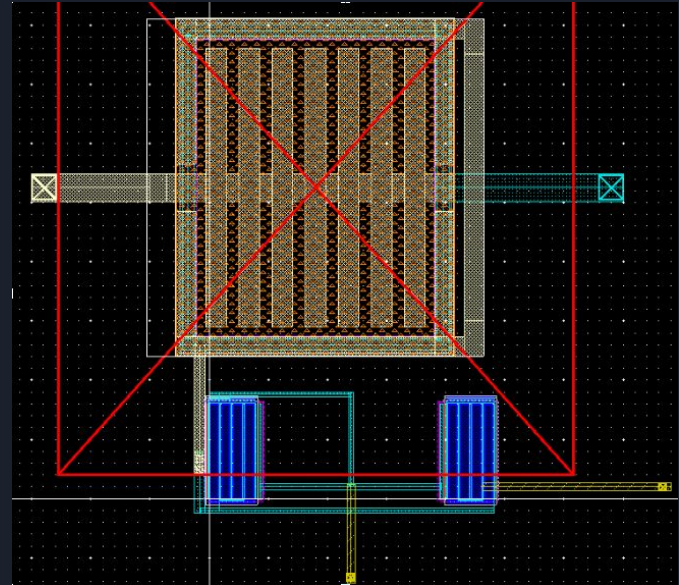
- In next slide i will show some examples to demonstrate the same...

Layout modification stage: one demonstration

LENGTH_OF_MOSFET=200n
WIDTH_OF_MOSFET=8u LENGTH_OF_CAP=25u
WIDTH_OF_CAP=20u NUMBER_OF_FINGERS=4



LENGTH_OF_MOSFET=800n
WIDTH_OF_MOSFET=8u LENGTH_OF_CAP=25u
WIDTH_OF_CAP=20u NUMBER_OF_FINGERS=4





Layout modification stage: Generalisation

- Just changing the params and distance between elements would give some degree of freedom, but still is very constrained.
- We need to generalise the layout formation such that, given the connections between components, the connections should form automatically without hardcoding the position of routing.
- For this we extend the terminals of the components and then consider the other end of extension as the actual ports (to avoid DRC errors of the metal wires being too close)

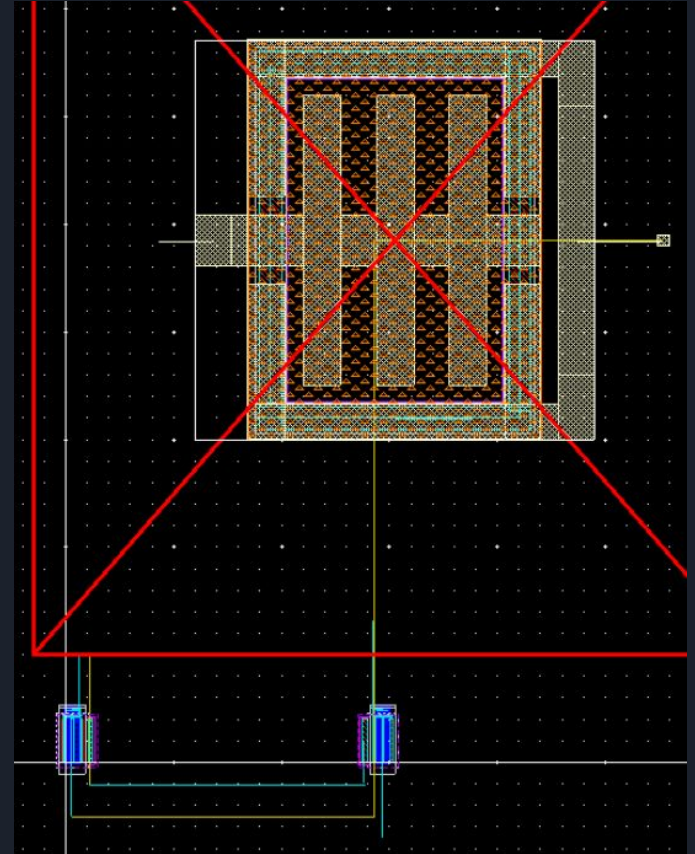


Layout modification stage: Generalisation

- A function has been implemented: `placeRobustRoute(cv [terminal1] [terminal2])` : this function takes in two terminals and input and connects them, by default with metal 1 as most connections in the layout are in metal 1.
- The speciality of this function is that it even checks if there are potential intersections of the metal wire with already present wiring and elements and moves the routing to higher metal layer.

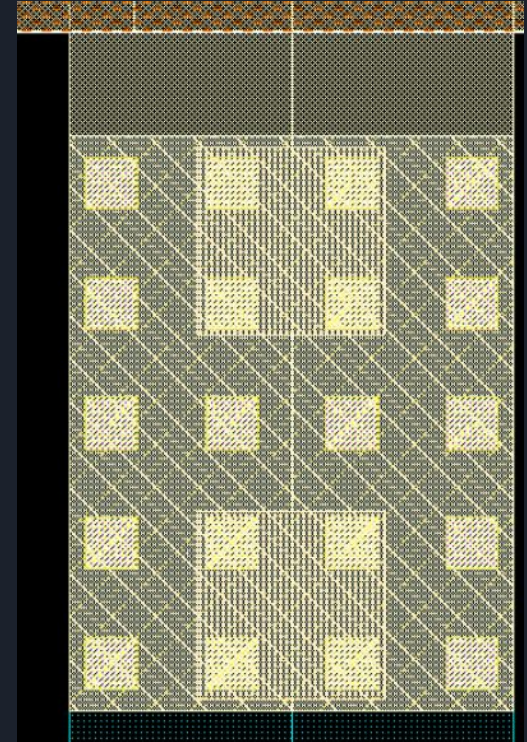
Layout modification stage: Generalisation

- On running the command:
`LENGTH_OF_MOSFET=200n`
`WIDTH_OF_MOSFET=8u`
`LENGTH_OF_CAP=25u`
`WIDTH_OF_CAP=20u`
`NUMBER_OF_FINGERS=4` virtuoso
`-nograph -restore`
`skill/create_layout_parameterised_noth`
`ardcoded.il` , we get the final layout like
this



Only complicated and semi-automated part: Via Creation

- Multiple vias need to be placed, also under certain arrangement to successfully be able to connect metals of different layers.
- Presently in the code, M2-M8 via is needed, and taking the via generated by GUI as reference, I have replicated via of similar setup at the required location.
- If, during generalisation over any schematic, vias connecting different metal layers are needed, the present code may not be able to support that.





Next steps and scope for improvement...

- More work can be done on routing scheme, and mainly on the via generation for multiple metal layer levels. To make this more generalisable for any given schematic, some work on via generation is needed as presently only M2-M8 via generation has been implemented
- The feedback loop is not set-up as every simulation for all configurations of layout I tried are giving the same PCE value, so feedback loop even if set-up, could not have been checked
- The extrusion width and wiring width has been hard-coded and it can be varied according to the layout requirements



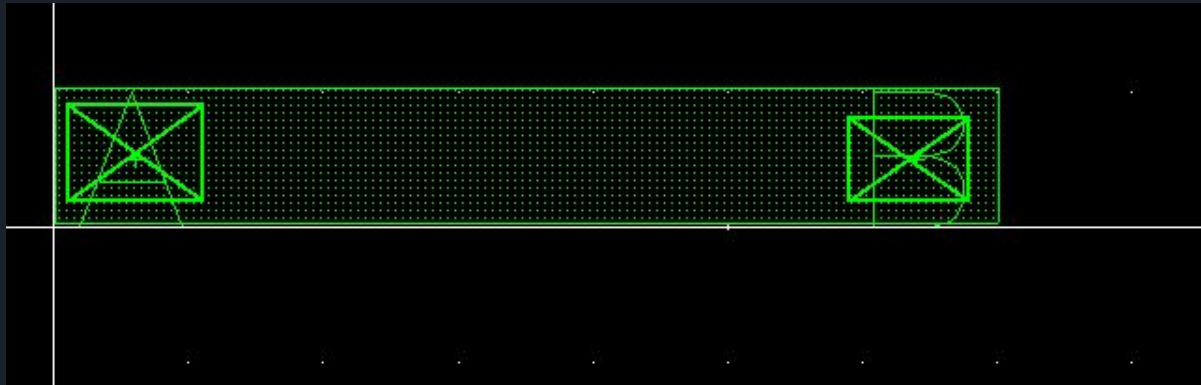
Just before we end... EMX!

- My BTP-1 project statement was to generate optimised inductor for given inductance value and Q-factor
- Electromagnetic extraction failed in my BTP - 1 because of which the scope of the project had to be changed
- Precisely, EMX was not able to detect ports and was throwing error that “port P000 not found”
- In the beginning of BTP -2, I first tried to pinpoint the issue in EMX so that anyone continuing my BTP-1 project would not need to struggle
- **The issue was that the process file provided by GPDK used in EMX can detect only upto metal layer 9 whereas my inductor was made up of metal 10 and metal 11**



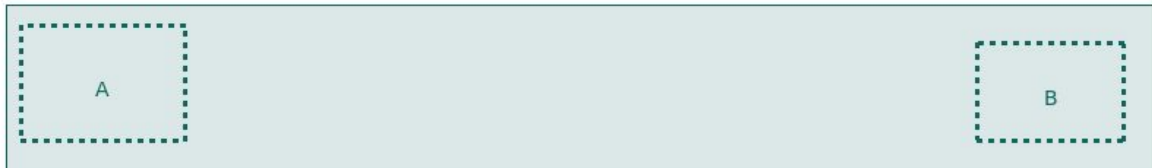
Just before we end... EMX!

To verify this I made a metal 7 layer with ports A and B of the same layer.



Just before we end... EMX!

We can see that EMX is able to detect the ports and the terminal does not throw error like last time



```
Done with frequency 1.000000e+08...
 4 9.81e-06 7.25e-06 [1]
 4 4.79e-06 3.37e-06 [2]
Done with frequency 2.000000e+10...
Done with frequency 1.005000e+10...
Reducing, current peak memory usage 112.71 MB...
Available reducing memory 3.95 GB, using b=10 for 8 active workers
Result playback...
Wrote /home/users/madhava/work_tsmc_gp_65_2f/EMX_work/trying_tsmc_trying_emx_using_bar.work/trying_emx_using_bar.y
Wrote /home/users/madhava/work_tsmc_gp_65_2f/EMX_work/trying_tsmc_trying_emx_using_bar.work/trying_emx_using_bar.s2p
CPU time 2.65 sec
Peak memory usage 112.71 MB
Wall-clock time 0.33 sec
Process exited normally
```



Thank You!